

Part 2: Reasoning-Based Questions

Author: Devanshu Shah
Email: devanshusah123499@gmail.com
Date: February 2026

Q1: Choosing the Right Approach

Scenario: Identifying whether a product is missing its label on an assembly line. The products are visually similar except for the label.

Question: Would you use classification, detection, or segmentation? Why? What would be your fallback if the first approach doesn't work?

Answer:

I would start with **object detection** as the primary approach, specifically detecting the presence or absence of the label as a distinct object. Detection is ideal here because it can localize where the label should be and determine if it's present, making it robust to variations in product positioning on the assembly line. Unlike classification, which only tells us "label present" or "label absent" for the entire image, detection provides spatial information about where the label is (or should be), which is valuable for quality control feedback. Additionally, detection handles cases where multiple products appear in the same frame, which is common in production environments.

If detection underperforms, my **fallback strategy** would be a two-stage approach: first use **segmentation** to identify the product boundary and expected label region, then apply **binary classification** on that cropped region to determine label presence. This fallback is more robust to label position variations and partial occlusions. Segmentation would help isolate the region of interest, removing background noise and focusing the classifier on the relevant area. I would also consider using **template matching** or **feature extraction** (SIFT/ORB) as a simpler backup if deep learning approaches fail due to limited training data, though this would be less robust to lighting and scale variations.

Another fallback worth considering is **anomaly detection**, where we train only on "good" (labeled) products and flag anything that deviates significantly as potentially missing a label. This approach works well when defect samples are rare and hard to collect. The key is to start with detection for its balance of accuracy and interpretability, then adapt based on real-world performance and failure modes observed in production.

Q2: Debugging a Poorly Performing Model

Scenario: You trained a model on 1000 images, but it performs poorly on new images from the factory.

Question: Design a small experiment or checklist to debug the issue. What would you test or visualize?

Answer:

My debugging approach would follow a systematic checklist to identify the root cause: **First, analyze the data distribution gap** by visualizing sample images from both training and production sets side-by-side. I would check for differences in lighting conditions, camera angles, image resolution, background clutter, and product positioning. Often, poor generalization stems from training data that doesn't represent production reality—for example, if training images were taken in controlled lighting but production has varied lighting throughout the day. I would create confusion matrices and plot confidence score distributions to see if the model is consistently uncertain or if errors are concentrated in specific classes.

Second, perform error analysis by manually reviewing 50-100 misclassified production images. I would categorize failures into buckets: blur/motion artifacts, occlusion, extreme lighting, unusual poses, or truly ambiguous cases. This reveals patterns—for instance, if 80% of errors occur with motion blur, the solution is to add augmented blurry images to training data. I would also visualize activation maps (Grad-CAM) to verify the model is looking at relevant features rather than spurious correlations like background elements or timestamps.

Third, validate the training process by checking learning curves for signs of overfitting (large train-val gap) or underfitting (both high), examining data augmentation adequacy, and verifying class balance. I would run the model on the original training images to ensure it still performs well, ruling out model corruption. Finally, I would compute domain-specific metrics like per-class precision/recall, worst-case error rates, and create a calibration plot to check if confidence scores are reliable. If production images are consistently darker, I'd test with brightness adjustments to quantify the lighting sensitivity gap.

Q3: Accuracy vs Real Risk

Scenario: Your model has 98% accuracy but still misses 1 out of 10 defective products.

Question: Is accuracy the right metric in this case? What would you look at instead and why?

Answer:

Accuracy is not the right metric in this scenario because it's highly misleading when dealing with imbalanced datasets, which is typical in defect detection where defective products are rare. With 98% accuracy, the model could simply predict "not defective" for every product and still achieve high accuracy if defects are only 2% of products. The critical insight is that missing 1 out of 10 defects (90% recall on defects) represents a **significant business risk**—those missed defects reach customers, potentially causing recalls, safety issues, or brand damage that far outweighs the cost of false positives.

I would focus on recall (sensitivity) as the primary metric for the defective class, aiming for at least 95-99% to minimize missed defects. This metric directly answers the business question: "What percentage of actual defects do we catch?" Additionally, I'd examine **precision** to understand the false positive rate—how many good products are flagged as defects. While false positives waste inspection time, false negatives (missed defects) are typically much more costly. I would compute the **F-beta score** with $\beta=2$ or higher to weight recall more heavily than precision, reflecting the asymmetric cost structure.

Beyond single metrics, I'd analyze the **confusion matrix** to understand the full error profile and calculate **cost-weighted accuracy** using actual business costs (e.g., missed defect costs \$1000, false alarm costs \$10). I would also track **worst-case recall**—the recall on the hardest-to-detect defect types—since these are the most dangerous misses. Finally, setting operating points using **precision-recall curves** allows us to choose a threshold that achieves target recall (e.g., 98%) while minimizing false positives, which is far more actionable than overall accuracy.

Q4: Annotation Edge Cases

Scenario: You're labeling data, but many images contain blurry or partially visible objects.

Question: Should these be kept in the dataset? Why or why not? What trade-offs are you considering?

Answer:

Yes, these edge cases should generally be kept in the dataset, but with careful consideration of how they're handled. The key principle is that training data should reflect the distribution of real-world data the model will encounter in production. If the deployed model will regularly see blurry images (e.g., from motion blur in video frames or low-quality cameras) or partial occlusions (e.g., objects at frame edges or behind other objects), training without these cases will create a harmful train-test mismatch. Excluding them produces a model that's brittle and fails in common real-world scenarios.

However, **the annotation strategy matters greatly**. For partially visible objects, I would label them if at least 30-50% of the object is visible and the class is clearly identifiable, using a "truncated" or "occluded" flag if the framework supports it (like COCO format). For extremely blurry images where even humans can't confidently identify the object, I would either exclude them or create a separate "uncertain" validation set to track model behavior on ambiguous cases without penalizing training. The trade-off is between **model robustness** (handling imperfect inputs) versus **label noise** (incorrect labels degrading learning).

The trade-offs I'm considering include: (1) Training stability—too many ambiguous examples can confuse the model and slow convergence, (2) Generalization—models trained on clean data often fail on noisy real-world inputs, requiring this exposure, (3) Annotation cost—spending time labeling very poor quality images may not be worth it if they're rare, and (4) Evaluation fairness—keeping a small "hard examples" validation set separate from main metrics prevents penalizing the model on truly impossible cases. My strategy would be to keep most edge cases (70-80%), mark them with difficulty flags, and potentially use harder example mining during training to handle them specifically without overwhelming the model with noise. This balanced approach builds robustness while maintaining training signal quality.

Summary of Approach

Throughout these questions, my reasoning follows key principles: (1) **Business Context First** - Metrics and decisions must align with real-world costs and risks, (2) **Data-Centric Mindset** - Most model failures stem from data issues, not architecture, (3) **Systematic Debugging** - Use structured checklists and visualization to identify root causes, (4) **Robustness Over Perfection** - Models should handle real-world messiness, not just clean test sets, and (5) **Cost-Aware Decisions** - Different error types have different consequences that must guide metric selection. These principles reflect the practical realities of deploying computer vision in production environments where reliability and business impact matter more than academic benchmark scores.

Contact Information

Author: Devanshu Shah

Date: February 2026

Email: devanshusah123499@gmail.com

LinkedIn: [linkedin.com/in/devanshu-shah-b6a316262](https://www.linkedin.com/in/devanshu-shah-b6a316262)